

Phase E-5 — SCALING experiment: tree-search quality vs. node budget (s3e3)

Question. D-2's 22-node harness_v2 sweep on `playground-series-s3e3` (Employee Attrition, ROC-AUC) found AUC 0.841442 in 34s and still had visible headroom (four lineages plateaued but the search hit its 22-node cap before exhausting the idea space). This experiment extends the same search regime to an 80-evaluated-node budget and measures the score-vs-evaluations curve directly, to answer: where does the curve flatten, and do late backtracks ever pay off?

Contents

Design	1
Curve table (best-so-far AUC vs. evaluations)	2
Where did the curve flatten, and did late backtracks pay?	2
Cross-run calibration (this run + 10 prior harness_v2 single-tree runs)	3
Verdict for Stage-4 budgeting	4

Design

- **Fresh tree**, not a modification of D-2's: `competitions/playground-series-s3e3/experiments_tree_scale.json`, produced by the new `tree_search/run_s3e3_scale.py` (reuses `eval_s3e3.py` for every solo/blend evaluation). D-2's own `tree/script/cache` directory were never touched.
- **Identical root, 5-fold StratifiedKfold, and 6 first-generation solo lineage seeds + 1 blend seed** as D-2 — imported directly from `run_s3e3.py` so the configs are byte-identical. This made **cache reuse** possible: the 7 seed nodes' cached OOF vectors from D-2's `tree_search/cache_s3e3/` were digit-verified (AUC recomputed from the cached array and checked against D-2's stored score to 6 decimals) and copied into this run's own `tree_search/cache_s3e3_scale/` instead of being retrained.
- **Mutation policy**: D-2's per-lineage authored queues, extended with boundary-push and joint-move items (exploit phase, evals 1-40), then — once the budget crosses eval 40 — six new **EXPLORE-phase long-shot lineages** seeded directly off the root: `EXTREMEREK` / `XGBSHALLOWIDE` (parameter extremes beyond every exploit-phase boundary-push), `CATDEEP` (a deliberate re-test of the "CatBoost is structurally weak here" closed item), `FEATKITCHEN` (joint prune of every redundant column found across the whole exploit phase in one move), and `KITCHENBLEND` / `RANKKITCHEN` (dirichlet-k800 / rank-space blends of every solo node evaluated so far). Every node's mutation string carries a `[PHASE exploit|explore|seed]` tag.

- **Curve logging:** every evaluated node appends an entry to `tree["curve"] ( eval_index , node_id , lineage , phase , auc , cum_best_auc , is_new_global_best , wall_s , cum_wall_s )` — the table below is read directly off the tree JSON.
- **Implementation note:** `harness_v2` 's child-dedup rejects a duplicate config but never consumes a `MAX_CHILDREN_PER_NODE` slot. Once the two kitchen-sink blend lineages had absorbed the entire solo pool, their `blend_fallback` "pool exhausted" branch started regenerating an identical config every call, and `select_next_parent` kept re-picking that same dead parent — an infinite loop that stalled the first run at 50/80 evaluated nodes. Fix: after 2 consecutive dedup rejections against the same parent, the driver burns a `status="failed"` placeholder child so the parent naturally hits its children cap and the search moves on (15 such placeholders exist in the final tree, all under the two kitchen-sink lineages — the exploit-phase per-model queues never hit this). The run was resumed cleanly after the fix; total real wall-clock across both process invocations was ~97s (32.9s to reach eval 40, ~65s more to reach eval 80 — the explore phase is more expensive per node: native-CatBoost and 30+-member kitchen-sink weight searches cost more than the lean 3-6-member exploit-phase blends).

Curve table (best-so-far AUC vs. evaluations)

evals	node (lineage, phase)	this node's AUC	cumulative best AUC
5	#4 (CATNATIVE, seed)	0.814259	0.837305
10	#9 (BLEND, exploit)	0.839441	0.839540
15	#14 (FEAT, exploit)	0.839821	0.842360
20	#19 (XGBTUNED, exploit)	0.838602	0.842360
30	#29 (CATORIG, exploit)	0.761395	0.842360
40	#39 (BLEND, exploit)	0.842766	0.843524
60	#74 (XGBTUNED, explore)	0.819722	0.845051
80	#94 (CATDEEP, explore)	0.800860	0.845051

Final best: **AUC 0.845051** (node #55, KITCHENBLEND lineage, explore phase) — **+0.003609 over D-2's 0.841442**, +0.006911 over the linear-iteration best (0.838140).

Where did the curve flatten, and did late backtracks pay?

All 10 global-best improvements over the whole run:

eval#	node	lineage	phase	AUC
1	#0	ROOT	seed	0.837305
7	#6	FEAT	seed	0.838903

eval#	node	lineage	phase	AUC
8	#7	BLEND	seed	0.839540
12	#11	FEAT	exploit	0.841442 (= D-2's best)
14	#13	FEAT	exploit	0.842360
38	#37	BLEND	exploit	0.843483
39	#38	BLEND	exploit	0.843524
45	#44	KITCHENBLEND	explore	0.844394
49	#48	KITCHENBLEND	explore	0.845034
52	#55	KITCHENBLEND	explore	0.845051

Nodes-per-improvement: evals 1-20 → 5 improvements (4.0 nodes/improvement); evals 21-40 → 2 improvements (10.0 nodes/improvement); evals 41-80 ("last 40") → 3 improvements, all concentrated in evals 41-52 (13.3 nodes/improvement over the full window, but effectively 4 nodes/improvement within the active sub-window, then **zero improvements in the final 28 evals**, 53-80).

Late-backtrack payoff — yes, decisively. All 3 post-eval-40 improvements (+0.001527 AUC beyond the exploit-phase ceiling of 0.843524) came from the SAME lineage: `KITCHENBLEND`, an explore-phase kitchen-sink blend (dirichlet, k=800) of every solo node found so far — including CatBoost variants and boundary-pushed models the exploit-phase per-model queues had already individually rejected. None of the other five explore lineages (`EXTREMEREGR` , `XGBSHALLOWWIDE` , `CATDEEP` , `FEATKITCHEN` , `RANKKITCHEN`) ever beat the exploit-phase ceiling — in particular, `CATDEEP` 's deliberate contradiction of the D-2 "CatBoost is structurally weak here" closed item (P3/P4) failed (best AUC ≈ 0.826 , well below LGB), **confirming** that prior verdict rather than overturning it. The exploit phase itself also had a long quiet stretch — 24 evals with zero improvement (eval 15→38) — that preceded a genuine within-exploit gain (eval 38/39), so idle stretches inside the exploit phase are not rare either.

Cross-run calibration (this run + 10 prior harness_v2 single-tree runs)

run	n_evaluated	best found at eval#	best/total ratio	idle tail
s3e1	22	14	0.64	8
s3e3 (D-2)	22	12	0.55	10
s3e5 v1	40	12	0.30	28
s3e5 v2	22	18	0.82	4
s3e7	22	14	0.64	8
s3e9 v1	20	2	0.10	18
s3e9 v2	26	14	0.54	12
s3e11	24	21	0.88	3

run	n_evaluated	best found at eval#	best/total ratio	idle tail
s3e14	20	12	0.60	8
s3e16	24	16	0.67	8
s3e19	22	18	0.82	4
s3e20	38	38	1.00	0
s3e3 scale (this run)	80	52	0.65	28

Mean best/total ratio ≈ 0.63 (range 0.10-1.00); mean idle tail ≈ 9.25 evals (range 0-28, prior max = s3e5 v1's 40-node run — the same 28-eval idle tail recurs at 2x the scale in this run, suggesting it's a real, not budget-specific, pattern for this harness).

Verdict for Stage-4 budgeting

1. **Default node budget: ~55-60 evaluated nodes**, structured as a fixed **exploit phase of ~35-40 evals** (this run's exploit-phase gains all landed by eval 39; cutting shorter than ~35 risks missing a real late-exploit gain like eval 38/39's, since idle stretches up to 24 evals occur inside the exploit phase itself) **followed by a mandatory, non-negotiable explore burst of 5-8 seeded long-shot lineages** (at minimum one kitchen-sink blend of every solo node found, plus 1-2 contrarian hyperparameter-boundary directions). The explore burst is not optional even if the exploit phase "looks done": it produced the single largest win of the whole run (+0.0015 AUC, +0.0036 over D-2) via an idea class (kitchen-sink ensembling) the exploit-phase per-model queues structurally cannot generate.
2. **Stopping rule: stop ~15-20 evals after the explore burst if none of them improved the global best.** This run's 3 late gains all landed within 12 evals of the phase switch (eval 41→52); the remaining 28 evals (53-80, 35% of the tested budget) produced zero further improvement. A "20 evals without improvement, measured only after the explore burst is injected" rule would have stopped at eval ≈ 72 , saving ~10% of this run's budget with no information loss. A flat non-improvement counter applied from the very start is NOT safe at this comp's scale — it would need patience $\geq 25-30$ to survive the 24-eval idle stretch inside the exploit phase alone, which leaves little room to save compute relative to the 80-node budget tested here.
3. **Numeric backstop:** hard-cap at 60 evaluated nodes regardless of the above if neither structural signal fires cleanly — consistent with the 10-prior-run mean best/total ratio of 0.63 and this run's own 0.65, with enough margin over both to almost never truncate a real improvement.